eg107.cpp

```
#include<iostream>
#include<thread>
using namespace std;
void do_something()
{
for(int i=201;i<=250;++i) cout<<i<<" ";
}
int main()
{
thread t(do_something);
for(int k=501;k<=550;++k) cout<<k<<" ";
return 0;
}
// the above application may crash
```

Thinking Machines Classroom Codes

eg108.cpp

```
#include<iostream>
#include<thread>
using namespace std;
void do_something()
{
for(int i=201;i<=250;++i) cout<<i<<" ";
}
int main()
{
thread t(do_something);
for(int k=501;k<=550;++k) cout<<k<<" ";
t.join();
return 0;
}
```

Thinking Machines Classroom Codes

eg109.cpp

```
#include<iostream>
#include<thread>
using namespace std;
void do_something(int p,int q)
{
for(int i=p;i<=q;++i) cout<<i<<" ";
}
int main()
{
thread t(do_something,100,200);
for(int k=501;k<=550;++k) cout<<k<<" ";
t.join();
return 0;
}
```

Thinking Machines Classroom Codes

eg110.cpp

```
#include<iostream>
#include<thread>
using namespace std;
void add(int p,int q,int &r) // this is incorrect, explicitly create a pointer
{
r=p+q;
}
int main()
{
int z;
thread t(add,10,20,z);
t.join();
cout<<z<<endl;
return 0;
}
```

Thinking Machines Classroom Codes

eg111.cpp

```
#include<iostream>
#include<thread>
using namespace std;
void add(int p,int q,int *r)
{
*r=p+q;
}
int main()
{
int z;
thread t(add,10,20,&z);
t.join();
cout<<z<<endl;
return 0;
}
```

eg112.cpp

```
#include<iostream>
#include<future>
#include<chrono>
#include<thread>
using namespace std;
int power(int n,int p)
{
if(p==0) return 1;
this_thread::sleep_for(chrono::seconds(1));
return (p==1)?n:n*power(n,p-1);
}
int main()
{
packaged_task task(power);
future<int> f=task.get_future();
cout<<"God is great 1234"<<endl;
task(5,3); // task is a callable object, separate thread comes in action over here
cout<<"God is great 5332"<<endl;
int r=f.get(); // join will get called for the separate thread which is behind the task
cout<<r<<endl;
return 0;
}
```

eg113.cpp

```
// data race scenario / shared resource between threads
#include<iostream>
#include<thread>
#include<chrono>
using namespace std;
struct cmn
{
int num;
};
void just_do_it(struct cmn *c,int num)
{
c->num=num;
cout<<c->num<<endl;
this_thread::sleep_for(chrono::seconds(1));
cout<<c->num<<endl;
}
int main()
{
struct cmn c;
thread t1(just_do_it,&c,10);
thread t2(just_do_it,&c,20);
thread t3(just_do_it,&c,30);
t1.join();
t2.join();
t3.join();
return 0;
}
```

Thinking Machines Classroom Codes

eg114.cpp

```
#include<iostream>
#include<thread>
#include<chrono>
#include<mutex>
using namespace std;
struct cmn
{
int num;
mutex mtx;
};
void just_do_it(struct cmn *c,int num)
{
lock_guard<mutex> lck(c->mtx);
c->num=num;
cout<<c->num<<endl;
this_thread::sleep_for(chrono::seconds(1));
cout<<c->num<<endl;
}
int main()
{
struct cmn c;
thread t1(just_do_it,&c,10);
thread t2(just_do_it,&c,20);
thread t3(just_do_it,&c,30);
t1.join();
t2.join();
t3.join();
return 0;
}
```

eg115.cpp

```
// sync is missing
#include<iostream>
#include<thread>
using namespace std;
struct cmn
{
int num;
};
void producer(struct cmn *c)
{
for(int i=501;i<=550;++i)
{
c->num=i;
cout<<"Produced : "<<c->num<<endl;
}
}
void consume(struct cmn *c)
{
int num;
for(int i=1;i<=50;i++)
{
num=c->num;
cout<<"Consumed : "<<num<<endl;
}
}
int main()
{
struct cmn c;
thread t1(producer,&c);
thread t2(consume,&c);
t1.join();
t2.join();
return 0;
}
```

eg116.cpp

```
#include<iostream>
#include<thread>
#include<mutex>
#include<functional>
#include<condition_variable>
using namespace std;
struct cmn
{
int num;
bool produced;
mutex mtx;
condition_variable cv;
};
bool should_produce(cmn *c)
{
return c->produced==false;
}
bool should_consume(cmn *c)
{
return c->produced==true;
}
void producer(struct cmn *c)
{
unique_lock<mutex> lck(c->mtx);
for(int i=501;i<=550;++i)
{
c->cv.wait(lck,bind(should_produce,c));
c->num=i;
cout<<"Produced : "<<c->num<<endl;
c->produced=true;
c->cv.notify_one();
}
}
void consume(struct cmn *c)
{
int num;
unique_lock<mutex> lock(c->mtx);
for(int i=1;i<=50;i++)
{
c->cv.wait(lock,bind(should_consume,c));
num=c->num;
cout<<"Consumed : "<<num<<endl;
c->produced=false;
c->cv.notify_one();
}
}
int main()
{
```

```
struct cmn c;
c.produced=false;
thread t1(producer,&c);
thread t2(consume,&c);
t1.join();
t2.join();
return 0;
}
```